

CodeLeWM: A Reproducible Code World-Model Study with Negative Action-Use Results and a Narrow Downstream Reranking Slice

Abdelhamid Bakhta
StarkWare

<https://github.com/AbdelStark/CodeLeWM>

Draft of June 8, 2026

Abstract

Latent world models are attractive for code because they promise to score candidate edits by predicting how a program state should change, rather than by directly generating patches. We built CodeLeWM, a manifest-backed research artifact that ports joint-embedding predictive architectures to Python code transitions and evaluates whether learned latent transitions improve candidate ranking. The main result is deliberately mixed: the original commit-edit action-use hypothesis is negative, with v0.2 text-action retrieval at Recall@1 0.263 and MRR 0.370 versus no-action Recall@1 0.441 and MRR 0.533, while the execution-trace substrate yields healthier diagnostics and a narrow downstream HumanEval WS-D replay where CodeLeWM reaches pass@1 0.9787 on two seeds and beats no-action by +10.64 and +8.51 points. The aggregate public model-quality claim remains closed because MBPP-Plus is saturated, with CodeLeWM, no-action, and lexical controls all at pass@1 1.0000; broad semantic surprise has no scorable semantic pairs; representation probes do not beat controls; and the final downstream rows do not expose a standalone `p_pass` score key. We therefore present CodeLeWM as a reproducible negative and diagnostic study with one narrow positive slice, not as evidence that the model generally improves coding. The paper’s contribution is the artifact and the lesson: code-world-model claims need no-action, lexical, manifest, and claim-gate discipline because many apparent wins vanish under those controls.

1 Introduction

World models are appealing for code because code edits have a natural transition structure: a before-state, an action or instruction, and an after-state. If a model can learn that transition, it may be useful even when it is not a code generator. A latent scorer could rank candidate patches produced by humans, search, or large language models, while keeping candidate code untrusted and avoiding execution in the scoring path.

Joint-embedding predictive architectures (JEPAs) learn by predicting future latents rather than reconstructing observations [2, 6]. LeWorldModel studies stable end-to-end JEPA-style world modeling from pixels [15]. CodeLeWM asks whether the same idea can be made credible for code edits. The transition contract is:

$$\hat{z}_{\text{after}} = P(E(s_{\text{before}}), A(a)). \quad (1)$$

The intended downstream use is ranking, not generation: given a before-state, an edit or execution action, and candidate after-states, score which candidate best matches the learned transition.

This paper reports the full evidence arc instead of selecting only the strongest slice. The early commit-message-conditioned substrate fails the action-use gate. A later execution-trace substrate repairs collapse and produces useful diagnostics. The final v0.9/v1.0 downstream replay shows a narrow HumanEval WS-D win but closes the aggregate claim because MBPP-Plus is solved by controls. This is a serious publishable result if framed correctly: not “we built a coding model that works,” but “we built a reproducible code world-model artifact, found where the hypothesis fails, isolated a narrow positive slice, and turned the failure modes into concrete next research requirements.”

Research questions.

1. **RQ1: Does action conditioning improve code-transition retrieval on commit-edit data?** No. The v0.2 action-swap intervention loses to no-action on the headline retrieval gate.
2. **RQ2: Can an execution-trace substrate support non-collapsed learned diagnostics?** Yes, in the narrow diagnostic sense. v0.9 retrieval beats retrieval controls, and mutation surprise AUC is 1.0 on both seeds.
3. **RQ3: Does the learned world model improve downstream candidate reranking?** Partially. HumanEval WS-D improves over no-action on two seeds, but MBPP-Plus is saturated by no-action and lexical controls, so the aggregate downstream claim is closed.

Contributions.

1. A reproducible code world-model artifact with dataset construction, training, retrieval, surprise, latent-probe, downstream reranking, paper-demo replay, manifests, checksums, and secret scans.
2. A negative action-use result showing that commit-edit action text does not beat a no-action baseline under the tested intervention.
3. A diagnostic execution-substrate study showing non-collapse, retrieval lift over controls, and mutation-surprise separation without overstating semantic representation claims.
4. A final downstream replay showing HumanEval WS-D lift over no-action on two seeds while closing the aggregate claim on MBPP-Plus saturation, missing standalone `p_pass` scores, and broader representation and semantic gates.
5. A concrete research outlook for making future code-world-model claims stronger: harder action-discriminative data, non-saturated downstream labels, first-class no-action and lexical controls, and explicit claim-gate artifacts.

2 Artifact and Claim Discipline

CodeLeWM is intentionally packaged as a research artifact. Each publishable surface writes a schema-versioned report, manifest, checksum evidence, and secret-scan result. Candidate code, generated patches, configs, checkpoints, and provider outputs are treated as untrusted inputs. The model-scoring path does not import, execute, or test-run candidate patches. Execution labels used for training or benchmark construction are produced in bounded sandboxed data build steps and then consumed as recorded artifacts.

Table 1: Evidence arc and public claim status. Each milestone is useful, but only the listed claims are open.

Milestone	Main evidence	Open claim	Closed or blocked claim
v0.2	action-swap A10G run	systems pipeline	action-use, latent axes, downstream utility
v0.6	execution substrate	diagnostics	stable downstream HumanEval/MBPP utility
v0.8	execution repair	HumanEval WS-D slice	MBPP, magnitude probe, broad semantic gate
v0.9	two-seed final gate suite	HumanEval WS-D slice	MBPP, representation, broad semantic gate
v1.0	paper-demo replay	reproducible demo package	aggregate downstream usefulness

The final paper-demo contract is deterministic. It replays checked-in v0.9 WS-D completion labels and score rows for four slices: seed 42 and seed 1729 crossed with HumanEval WS-D and MBPP-Plus WS-D. Replay mode records `score_source=replay_existing_scores`; it is not a fresh checkpoint scoring run and not a live OpenRouter demo. The committed artifact id is `demo_report-e6fc06c328eed245`.

The release includes a machine-readable claim gate. That gate is not a decorative checklist; it is part of the artifact contract. It allows narrow HumanEval WS-D wording and blocks aggregate coding-improvement wording because the agreed cross-benchmark gate does not pass.

3 Experimental Evidence

The final claim audit is `docs/benchmark/V1_0_FINAL_CLAIM_AUDIT_2026-06-08.md`. It consolidates v0.2, v0.6, v0.8, v0.9, and v1.0 paper-demo evidence. Table 1 summarizes the arc.

3.1 RQ1: negative action-use evidence

The v0.2 action-swap/inverse-action run directly tests whether action conditioning beats no-action on held-out retrieval. It does not. Text-action Recall@1 was 0.263 and MRR was 0.370048, while no-action reached Recall@1 0.441 and MRR 0.533105. Exact-same-before, near-before, same-file, action-cluster, edit-shape, and random action-contrast pools also failed to show the required margin over no-action. Latent probes were unsupported for semantic structure: lexical or metadata controls beat the latent views on the available targets.

This is the central negative result. It means that the early edit-action signal was not sufficiently action-discriminative. A commit message can describe a change, but it often does not force the model to use the action rather than the before-state. The no-action baseline is therefore not a nuisance baseline; it is a first-class scientific control.

3.2 RQ2: execution-substrate diagnostics

The execution substrate changes the state/action/next-state signal from noisy commit messages to deterministic execution triples. This improves training health and diagnostic separability. By v0.9, retrieval lift over controls passes on both seeds: Recall@1 is 0.2538 and 0.2731, while all listed

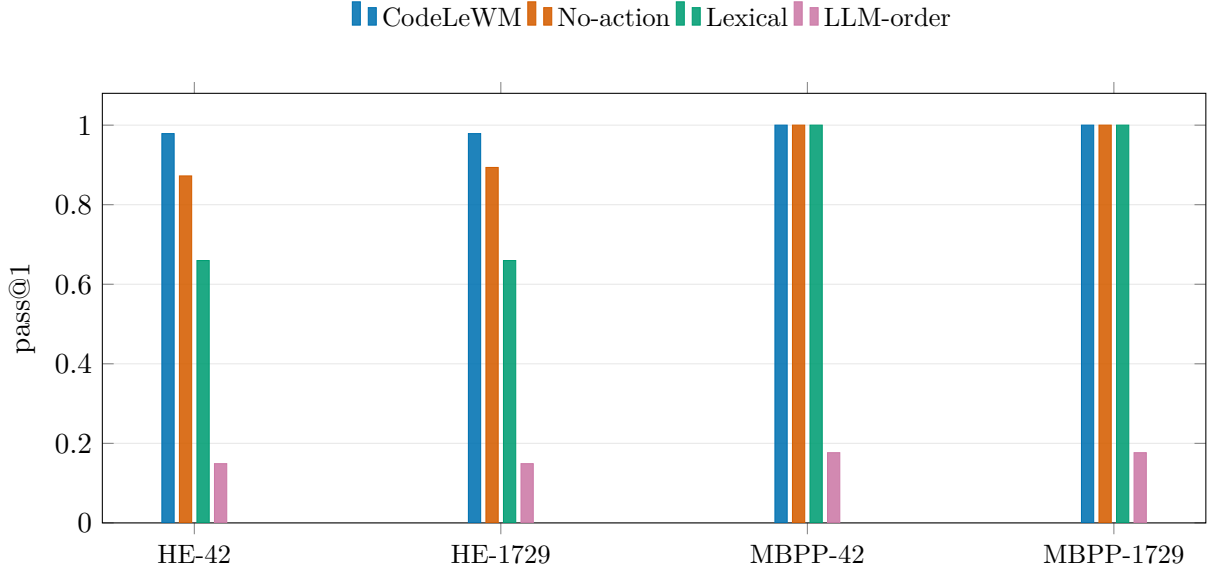


Figure 1: Final v1.0 paper-demo replay. HumanEval WS-D shows a narrow CodeLeWM lift over no-action, while MBPP-Plus is saturated by CodeLeWM, no-action, and lexical controls. This is why the aggregate claim gate is closed.

Table 2: Final v1.0 paper-demo replay. The aggregate claim gate is closed even though the HumanEval WS-D slices are open. Lift is pass@1 points over no-action.

Seed	Benchmark	CodeLeWM	No-action	LLM-order	Lexical	Lift	Gate
42	HumanEval WS-D	0.9787	0.8723	0.1489	0.6596	+10.64	open
42	MBPP-Plus WS-D	1.0000	1.0000	0.1765	1.0000	+0.00	closed
1729	HumanEval WS-D	0.9787	0.8936	0.1489	0.6596	+8.51	open
1729	MBPP-Plus WS-D	1.0000	1.0000	0.1765	1.0000	+0.00	closed

retrieval controls are below 0.024 Recall@1. Mutation surprise AUC is 1.0 on both seeds. These are credible training-health results. They do not, by themselves, prove downstream coding usefulness.

Representation claims remain closed. The v0.9 **passed** probe reaches 0.6552 and 0.6158 for $\hat{z}_{\text{after}}$, but no-action or before-state controls are stronger. The **output_magnitude_bucket** target is evaluable in v0.9 but tied with a direct **z_after** control. Broad semantic surprise also remains closed because the semantic-decoy categories have zero scorable aligned pairs in the final pack.

3.3 RQ3: final downstream replay

The final paper-demo replay is the central downstream result. It is derived from `docs/benchmark/v1_0/paper_demo/reports/paper_demo_report.json` and summarized in Figure 1 and Table 2.

HumanEval WS-D is the narrow positive slice: CodeLeWM reaches pass@1 0.9787 on both seeds and beats no-action by +10.64 and +8.51 points. The 95% bootstrap confidence intervals for lift over no-action are [2.13, 21.28] and [2.13, 17.02], respectively. LLM-order is much weaker at pass@1 0.1489.

MBPP-Plus is the counterweight. CodeLeWM reaches pass@1 1.0000, but so do no-action and lexical controls. The measured lift over no-action is exactly +0.00 points on both seeds with confidence interval [0.00, 0.00]. This closes the aggregate downstream claim. The evidence supports

a narrow HumanEval WS-D result, not a claim that CodeLeWM generally improves patch ranking.

4 Lessons

No-action is a scientific baseline, not a convenience baseline. Many code transitions are sparse. Large portions of the after-state are identical to the before-state, and many candidate completions can be ranked using surface similarity. A world model must learn when the action changes behavior, not merely when text resembles the prompt. The v0.2 failure shows that weak action text can leave no-action stronger than the learned transition.

Substrate health is not downstream utility. The execution substrate yields healthier retrieval and mutation diagnostics, but those diagnostics do not automatically imply useful patch ranking. This distinction matters. Non-collapse, retrieval lift over controls, and surprise separation are necessary health signals, not sufficient product claims.

Benchmark saturation can close an otherwise promising result. The final WS-D MBPP-Plus pack is saturated. CodeLeWM, no-action, and lexical all choose passing candidates at rank 1 on both seeds. Saturation prevents a lift claim because the model did not add measurable value beyond simple controls. This is not a failure of the replay command; it is precisely why the claim gate exists.

Representation claims require their own evidence. Healthy training diagnostics do not justify naming semantic latent axes. Frozen probes must beat controls across targets and seeds, and semantic-decoy surprise must have enough scorable aligned pairs. In the final evidence, those conditions do not hold.

5 Related Work

Joint-embedding prediction. I-JEPA and V-JEPA learn feature predictions without pixel reconstruction [2, 6]. LeWorldModel studies stable end-to-end JEPA-style world modeling from pixels [15]. Anti-collapse regularization is central to this family, with related ideas in VICReg, Barlow Twins, and LeJEPA [4, 5, 19].

Code modeling and execution signals. Pretrained code models such as CodeBERT, GraphCodeBERT, UniXcoder, and CodeT5 learn program representations or generation models from code and natural language [10–12, 17]. HumanEval, MBPP, APPS, CodeNet, and EvalPlus-style benchmark extensions provide candidate tasks and labels for coding evaluation [3, 8, 13, 14, 16]. Generated tests have also been used for reranking code generations [7]. Execution-aware pre-training and execution-trace work provide complementary signals for program behavior [1, 9, 18]. CodeLeWM differs by treating code changes and execution records as manifested latent transitions and by making no-action, lexical, manifest, and claim-gate controls part of the artifact.

6 Reproducibility

The source repository is <https://github.com/AbdelStark/CodeLeWM>. The main artifact reports used in this paper are:

- docs/benchmark/V0_2_ACTION_SWAP_HF_RESULTS_2026-05-20.md
- docs/benchmark/EXECUTION_V0_8_RESULTS_2026-06-05.md
- docs/benchmark/EXECUTION_V0_9_RESULTS_2026-06-07.md
- docs/benchmark/PAPER_DEMO_V1_0_ARTIFACTS_2026-06-08.md
- docs/benchmark/V1_0_FINAL_CLAIM_AUDIT_2026-06-08.md
- docs/benchmark/v1_0/paper_demo/manifest.json

The deterministic replay command for the final demo is:

```
uv run scripts/paper-demo --out docs/benchmark/v1_0/paper_demo --overwrite --json
```

The paper build command is:

```
scripts/build-codelewm-final-paper
```

The final release reproducibility checklist is docs/release/V1_0_REPRODUCIBILITY_CHECKLIST_2026-06-08.md.

7 Limitations

- The final positive result is a slice result, not a broad benchmark result.
- The final paper demo replays checked-in v0.9 score rows. It is reproducible and manifest-backed, but it is not a fresh checkpoint scoring run.
- MBPP-Plus saturation prevents measuring lift on that slice.
- The representation probes do not support naming semantic dimensions.
- Broad semantic surprise remains blocked by scorable-pair coverage.
- Candidate code remains untrusted; the model-scoring path does not execute candidate patches.
- The results do not compare against large pretrained code models as generators. CodeLeWM is a scorer/reranker artifact.

8 Outlook: What Would Make the Next Claim Stronger?

The next credible positive claim should be designed around gates that would change the conclusion, not around prettier demos. We see five concrete directions.

Action-discriminative data. The training data should contain same-before and near-before candidate sets where the action is necessary to choose the target. This directly attacks the v0.2 no-action failure.

Non-saturated downstream labels. A future downstream benchmark should contain at least 100 public-safe labeled tasks per target slice, with no-action and lexical controls below saturation. The claim should require lift over no-action, lexical, shuffled-action, and LLM-order baselines with confidence intervals excluding zero.

First-class calibrated scoring heads. If `p_pass` is intended to support downstream ranking, it should be serialized as a distinct score key in every downstream row, calibrated on held out tasks, and evaluated separately from energy-derived CodeLeWM scores.

Representation probes tied to executable semantics. Semantic latent-axis claims should require probes that beat before-state, no-action, lexical, and direct `z_after` controls across seeds. Broad semantic surprise should require enough scorable pairs to make the gate meaningful.

Live utility only after offline gates. LLM patch demos are useful workflow evidence, but live utility claims should come after offline gates pass. Candidate code should remain untrusted, and any execution should stay inside explicit sandbox allowlists with manifests and secret scans.

9 Conclusion

CodeLeWM is a completed reproducible research artifact, not a broad coding assistant. The artifact discipline worked: datasets, checkpoints, reports, claim gates, manifests, and scans can be regenerated and inspected. The original action-use hypothesis did not work: no-action stayed stronger on commit-edit retrieval. The execution substrate gave healthier model diagnostics and a narrow HumanEval WS-D downstream win, but MBPP-Plus saturation and representation blockers keep the public model-quality claim closed. The most useful conclusion is methodological: code world-model claims should be published with their baselines and failure gates intact, especially when no-action and lexical controls are strong.

References

- [1] Jordi Armengol-Estap, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael F. P. O’Boyle, Sida Wang, Gabriel Synnaeve, and Hugh James Leather. What i cannot execute, i do not understand: Training and evaluating llms on program execution traces, 2025. URL <https://arxiv.org/abs/2503.05703>.
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL <https://arxiv.org/abs/2301.08243>.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [4] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL <https://arxiv.org/abs/2511.08544>.
- [5] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022. URL <https://arxiv.org/abs/2105.04906>.
- [6] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL <https://arxiv.org/abs/2404.08471>.

- [7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022. URL <https://arxiv.org/abs/2207.10397>.
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [9] Yangruibo Ding, Ben Steenhoeck, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. Traced: Execution-aware pre-training for source code, 2023. URL <https://arxiv.org/abs/2306.07487>.
- [10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages, 2020. URL <https://arxiv.org/abs/2002.08155>.
- [11] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. Graphcodebert: Pre-training code representations with data flow, 2021. URL <https://arxiv.org/abs/2009.08366>.
- [12] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation, 2022. URL <https://arxiv.org/abs/2203.03850>.
- [13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps, 2021. URL <https://arxiv.org/abs/2105.09938>.
- [14] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. URL <https://arxiv.org/abs/2305.01210>.
- [15] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. URL <https://arxiv.org/abs/2603.19312>.
- [16] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks, 2021. URL <https://arxiv.org/abs/2105.12655>.

- [17] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021. URL <https://arxiv.org/abs/2109.00859>.
- [18] Wojciech Zaremba and Ilya Sutskever. Learning to execute, 2015. URL <https://arxiv.org/abs/1410.4615>.
- [19] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction, 2021. URL <https://arxiv.org/abs/2103.03230>.